



**EJECUTAR**



SESIÓN 05 · UNIDAD II · ETHEREUM Y CONTRATOS

# ETHEREUM Y LA MÁQUINA VIRTUAL

Un computador que nadie apaga, que todos ejecutan a la vez, y en el que cada instrucción tiene precio.

Blockchain y Web 3.0 · Ingeniería de Sistemas  
Universidad de San Buenaventura Medellín

# CÓMO VAMOS A USAR LAS TRES HORAS

Sesión de transición: se cierra el caso Estonia y empieza la Unidad II, la columna vertebral del curso.

|          |                                                                                            |         |
|----------|--------------------------------------------------------------------------------------------|---------|
| <b>E</b> | <b>EXPOSICIONES · CASO ESTONIA</b><br>Cinco minutos por equipo, preguntas y síntesis.      | ~45 min |
| <b>A</b> | <b>LA MÁQUINA DE ESTADOS</b><br>Cuentas, transacciones, gas, EIP-1559 y la EVM por dentro. | ~60 min |
| <b>B</b> | <b>LABORATORIO 05</b><br>Análisis forense de transacciones reales y medición de gas.       | ~60 min |
| <b>C</b> | <b>CIERRE</b><br>Evidencia, pregunta de la semana y lo que trae Solidity.                  | ~15 min |



# AL SALIR DE AQUÍ

**Entender Ethereum como máquina de estados replicada y leer en un explorador el costo real de una transacción.**

## LAS PREGUNTAS QUE LA SESIÓN RESPONDE

- 01** ¿Qué cambia al pasar de un registro de pagos a un computador compartido?
- 02** ¿Qué contiene exactamente una transacción, y qué hace cada campo?
- 03** ¿Por qué cada instrucción cuesta gas, y cómo se fija su precio?
- 04** ¿Por qué escribir un dato cuesta cien veces más que sumar dos números?



# SIGLAS Y TÉRMINOS DE HOY

**EVM***Ethereum Virtual Machine*

La máquina virtual que ejecutan todos los nodos. Mismo programa + mismo estado = mismo resultado en todos.

**UTXO***Unspent Transaction Output*

Salida de transacción no gastada: el modelo de Bitcoin, sin saldos, solo «billetes» por gastar.

**GAS***unidad de trabajo computacional*

Cada instrucción consume una cantidad fija. Se paga en ETH según el precio del momento.

**OPCODE***código de operación*

Instrucción elemental de la EVM: ADD, SLOAD, SSTORE, CALL... Cada una con su costo en gas.

**EOA***Externally Owned Account*

Cuenta controlada por una clave privada. Es la única que puede iniciar una transacción.

**WEI · GWEI***unidades de ether*

1 ETH =  $10^{18}$  wei. 1 gwei =  $10^9$  wei. Todo se calcula en enteros: no hay decimales en la EVM.

**EIP-1559***Ethereum Improvement Proposal 1559*

Mecanismo de tarifa desde 2021: una tarifa base que se quema más una propina para el validador.

**ABI***Application Binary Interface*

La convención para codificar qué función se llama y con qué argumentos dentro del campo data.



APROXIMADAMENTE 45 MINUTOS

# EXPOSICIONES · EL CASO ESTONIA

Cinco minutos por equipo. Veredicto al frente, la mejor evidencia en pantalla, y la fuente abierta para responder.

# CÓMO CORREN LAS EXPOSICIONES

- |           |                      |                                                                                                                          |
|-----------|----------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>01</b> | <b>ORDEN</b>         | Por sorteo, no voluntario. Se anuncia el orden completo antes de empezar para que nadie se prepare mientras otro expone. |
| <b>02</b> | <b>CINCO MINUTOS</b> | Cronómetro visible. Al minuto 5 se corta, sin excepción: elegir qué mostrar es parte de lo que se evalúa.                |
| <b>03</b> | <b>UNA PREGUNTA</b>  | Cualquier persona del curso pregunta de dónde salió un dato. El equipo abre la fuente y señala el lugar (E.3).           |
| <b>04</b> | <b>REGISTRO</b>      | Mientras un equipo expone, los demás anotan su veredicto en una línea. Lo usamos en la síntesis.                         |

## LO QUE SE EVALÚA EN ESTE MOMENTO

El 10 % de exposición de la rúbrica. El documento escrito se califica aparte.

# QUÉ SE ESPERA EN CADA MINUTO, Y CÓMO SE CALIFICA

| TIEMPO      | QUÉ MUESTRAN                                             | QUÉ DISTINGUE UNA BUENA EXPOSICIÓN                                                 |
|-------------|----------------------------------------------------------|------------------------------------------------------------------------------------|
| 0:00 – 0:30 | El veredicto: sí, no, o depende de una definición dicha. | Se dice de entrada, sin suspenso. «Depende» sin definición no es veredicto.        |
| 0:30 – 1:30 | La definición operativa de «cadena de bloques».          | Propiedades verificables una por una, no una frase de diccionario.                 |
| 1:30 – 3:30 | La mejor evidencia: UNA, en pantalla.                    | Captura con fecha, cita literal o comparación de versiones que se puede localizar. |
| 3:30 – 4:15 | El límite: qué NO pueden concluir.                       | No confundir «el folleto exagera» con «la tecnología no sirve».                    |
| 4:15 – 5:00 | La proyección (T6): un problema de su entorno.           | Enunciado sin nombrar la tecnología.                                               |

| CRITERIO DEL 10 %       | 5,0                                         | 3,0                                   | 0                  |
|-------------------------|---------------------------------------------|---------------------------------------|--------------------|
| Tiempo y estructura     | Veredicto antes de 0:30 y termina a tiempo. | Veredicto tardío o se corta a medias. | Sin veredicto.     |
| Respuesta a la pregunta | Abre la fuente y señala el lugar exacto.    | Encuentra la fuente, no el lugar.     | No la puede abrir. |



# ¿DE DÓNDE SALIÓ ESE DATO?

**01**

LA PREGUNTA

Una sola, y siempre sobre un dato concreto: «el conteo de la tarea T1», «la fecha de la captura», «esa cita del artículo». No se vale preguntar opiniones.

**02**

60 SEGUNDOS

El equipo abre la fuente en pantalla y señala el lugar: página, párrafo, captura con fecha. Puede responder cualquier integrante.

**03**

SI LO ENCUENTRA

Se anota «evidencia localizada». Es lo que hace creíble todo lo demás que dijeron.

**04**

SI NO LO ENCUENTRA

Se anota «no localizada» y el docente revisa ese punto en el documento: pesa en el 30 % de evidencia verificable.



LA ÚNICA PREGUNTA QUE HACE SIEMPRE EL DOCENTE

«¿Qué NO pueden concluir con esa evidencia?» Se responde en una frase. Es el criterio de honestidad intelectual.



# LO QUE QUEDÓ DEMOSTRADO Y LO QUE NO

## HECHOS VERIFICABLES

- El whitepaper de Bitcoin no contiene la palabra «blockchain».
- El FAQ oficial reconoce que se llamaba «hash-linked time-stamping».
- El artículo de Guardtime ancla la confianza en publicaciones periódicas en medios, incluidos periódicos.
- Los propios autores citan a Haber y Stornetta como origen de la técnica.

## JUICIOS QUE DEPENDEN DE LA DEFINICIÓN

- Si «cadena de bloques» exige participación abierta y consenso entre desconocidos, KSI no lo es.
- Si basta con encadenamiento criptográfico verificable, sí lo es.
- En ningún caso el error del folleto prueba que KSI sea mala tecnología.

**Un veredicto bien sostenido no es el que acierta: es el que dice qué definición usó y qué no puede concluir.**



# PREGUNTA DE VERIFICACIÓN

**Un equipo concluye: «como el FAQ oficial se equivocó sobre el whitepaper, el sistema KSI no es confiable». ¿Qué está mal en ese razonamiento?**

- A** Nada: si la fuente oficial se equivoca, todo el sistema queda en duda.
- B** Confunde un error en el material de mercadeo con una falla técnica del sistema.
- C** El FAQ no se equivocó: el whitepaper sí usa la palabra «blockchain».

Respuesta en voz alta y en una frase. Vale lo mismo que el párrafo del límite de la tarea T5.

# A



APROXIMADAMENTE 60 MINUTOS

# LA MÁQUINA DE ESTADOS

Ethereum no es una moneda con extras. Es un computador compartido que, además, tiene una moneda para cobrar su uso.

# DEL DINERO PROGRAMABLE AL COMPUTADOR COMPARTIDO

## BITCOIN · 2009

- Un lenguaje de guion deliberadamente limitado: sin bucles.
- Pensado para una sola cosa: condiciones de gasto de monedas.
- Limitarlo es una decisión de seguridad, no un descuido.

## ETHEREUM · 2015

- Propuesto por Vitalik Buterin en 2013; red en marcha en julio de 2015.
- Un lenguaje de propósito general: Turing-completo.
- Cualquier programa, siempre que alguien pague cada paso que ejecuta.

**La pregunta que cambió: ya no es «¿quién le pagó a quién?», sino «¿cuál es el estado de todos los programas después de esta transacción?».**

Turing-completo trae un problema que Bitcoin evitaba: un programa puede no terminar nunca. Guarden esa idea: es la razón de ser del gas.



# MODELO UTXO FRENTE A MODELO DE CUENTAS

El mismo pago —Ana tiene 10 y le paga 7 a un comercio— registrado de las dos maneras:

## UTXO · BITCOIN · COMO BILLETES

No existe «el saldo de Ana». Existen salidas no gastadas que su clave puede desbloquear. Para pagar 7 con un billete de 10, se gasta el billete ENTERO y se crean dos salidas nuevas: 7 para el comercio y 3 de cambio para Ana.

BILLETE 10

7 → COMERCIO

3 → ANA

## CUENTAS · ETHEREUM · COMO UN BANCO

Existe un registro global: la dirección de Ana tiene 10. Pagar 7 resta 7 de su cuenta y suma 7 a la otra. Las cuentas de contrato, además, guardan código y datos propios.

ANA: 10 → 3

COMERCIO: 0 → 7

Error típico: creer que en Bitcoin «se descuenta del saldo». No hay saldo que descontar: el saldo que muestra una billetera es la suma de sus billetes sin gastar.



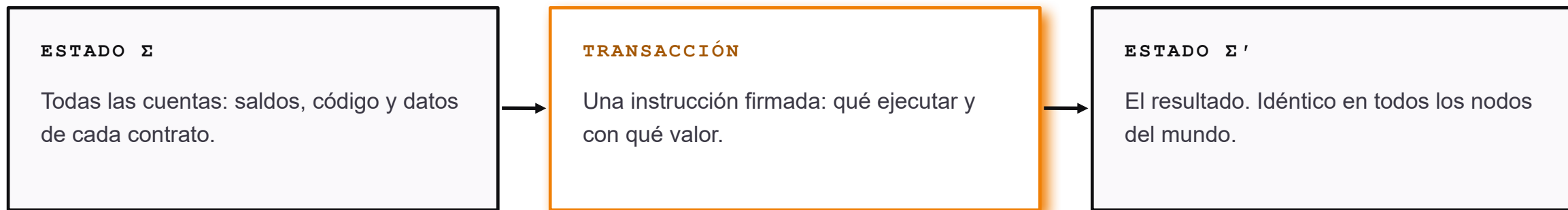
# NINGUNO ES MEJOR: RESUELVEN COSAS DISTINTAS

| CRITERIO                 | UTXO                                                                                | CUENTAS                                                            |
|--------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>Privacidad</b>        | Mejor: cada pago puede usar direcciones nuevas y no hay saldo acumulado a la vista. | Peor: una dirección concentra todo su historial y su saldo.        |
| <b>Paralelismo</b>       | Natural: dos transacciones que gastan salidas distintas no se estorban.             | Difícil: dos transacciones sobre la misma cuenta deben ordenarse.  |
| <b>Programabilidad</b>   | Limitada: el estado vive repartido en salidas sueltas.                              | Natural: un contrato tiene su saldo y su memoria en un solo lugar. |
| <b>Repetición</b>        | Imposible por diseño: una salida gastada ya no existe.                              | Exige el contador de cuenta (nonce) que vimos en la Sesión 3.      |
| <b>Tamaño del estado</b> | El conjunto de salidas no gastadas.                                                 | Todas las cuentas con todo su almacenamiento: crece sin parar.     |

El modelo de cuentas es la condición para que existan los contratos inteligentes tal como los vamos a programar. Es su mayor ventaja y la raíz de varios problemas que veremos en seguridad.

# UNA MÁQUINA DE ESTADOS REPLICADA

```
estado_nuevo = aplicar( estado_anterior , transacción )
```



Para que miles de nodos lleguen al mismo  $\sigma'$  sin comunicarse el resultado, la ejecución tiene que ser DETERMINISTA. Eso prohíbe cosas que en otros entornos son triviales:

- Leer la hora exacta del computador, un archivo local o una página web.
- Generar números aleatorios de verdad: la aleatoriedad en cadena es un problema abierto (Sesión 9).
- Usar punto flotante: dos procesadores podrían redondear distinto.



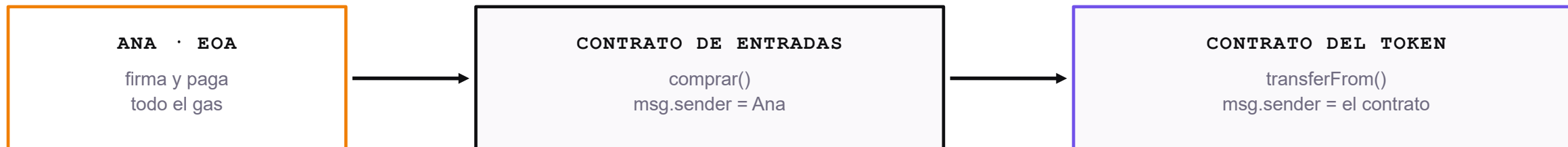
# CUENTAS EXTERNAS Y CUENTAS DE CONTRATO

|                      | CUENTA EXTERNA (EOA)                    | CUENTA DE CONTRATO                                   |
|----------------------|-----------------------------------------|------------------------------------------------------|
| La controla          | Quien tenga la clave privada.           | Su propio código. Nadie la controla desde fuera.     |
| Tiene código         | No.                                     | Sí: el bytecode desplegado, que no se puede cambiar. |
| Inicia transacciones | Sí. Es la única que puede.              | No. Solo reacciona cuando alguien la llama.          |
| Su dirección sale de | El hash de la clave pública (Sesión 3). | La dirección de quien la desplegó y su contador.     |
| Ejemplo              | Su billetera del curso.                 | El contrato de entradas del proyecto del docente.    |

## [ ] LA CONSECUENCIA QUE MÁS SORPRENDE

Un contrato nunca hace nada por sí solo. No hay tareas programadas ni procesos de fondo: todo empieza con una persona firmando una transacción.

# QUIÉN HACE QUÉ CUANDO ANA USA UN CONTRATO



- 01 UNA TRANSACCIÓN** Ana firma UNA transacción dirigida al contrato de entradas. Todo lo que pase después ocurre dentro de ella.
- 02 LLAMADA INTERNA** El contrato de entradas llama al del token. Eso no es una transacción nueva: es un mensaje dentro de la misma.
- 03 TODO O NADA** Si el token revierte, se deshace también lo que hizo el contrato de entradas. Ana paga el gas consumido igual.

## ⚠ ERROR TÍPICO

«El contrato me cobró». Un contrato no puede tomar nada que no le hayan enviado o autorizado antes.

# LOS CUATRO CAMPOS DEL ESTADO

**nonce**

Cuántas transacciones envió (EOA) o cuántos contratos creó (contrato). El contador anti-repetición de la Sesión 3.

**balance**

Su saldo de ETH en wei. Un entero de 256 bits.

**storageRoot**

La raíz de un árbol con todos los datos persistentes del contrato. Vacío en una EOA.

**codeHash**

El hash del código del contrato. En una EOA, el hash de «nada».



**ERROR TÍPICO: «MIS TOKENS ESTÁN EN MI CUENTA»**

No. El campo balance es solo ETH. Sus 25 LINK son una fila en el storage del CONTRATO de LINK: «dirección de ustedes → 25». Por eso transferir un token es llamar a ese contrato, y por eso el Value sale en 0 (lo verán en el laboratorio).



# NO HAY DECIMALES: TODO SE CUENTA EN WEI

| UNIDAD       | EN WEI                                  | PARA QUÉ SE USA                                                         |
|--------------|-----------------------------------------|-------------------------------------------------------------------------|
| <b>wei</b>   | 1                                       | La unidad real. Todo saldo y todo cálculo dentro de la EVM está en wei. |
| <b>gwei</b>  | 1 000 000 000 ( $10^9$ )                | Precios del gas. «La tarifa base está en 1 gwei».                       |
| <b>ether</b> | 1 000 000 000 000 000 000 ( $10^{18}$ ) | Montos que ve una persona. Solo existe en las interfaces.               |

Por qué enteros: la EVM tiene que dar el mismo resultado en todos los procesadores (A.4). El punto flotante redondea distinto según el hardware; los enteros no. Los tokens copian la idea: USDC usa 6 decimales, LINK usa 18.

```
// JavaScript con ethers v6
ethers.parseEther("0.5")           // 500000000000000000n
ethers.formatEther(500000000000000000n) // "0.5"
ethers.parseUnits("3", "gwei")      // 3000000000n
```



# QUÉ CONTIENE EXACTAMENTE LO QUE SE FIRMA

TRANSACCIÓN REAL · SEPOLIA · BLOQUE 11 727 244

```
{
  type:                2,           // EIP-1559
  chainId:             11155111,    // Sepolia
  nonce:               1820,
  maxPriorityFeePerGas: 2000000000,  // 2 gwei
  maxFeePerGas:        4051050452,  // ≈ 4,05 gwei
  gasLimit:            21000,
  to:                  "0xCA0e...a684",
  value:               11730000000000, // 0,0001173 ETH
  data:                "0x",
  accessList:          [],
  // firma: v (yParity), r, s
}
```

## CHAINID

Impide repetir la firma en otra red. Una transacción de Sepolia no vale en la red principal.

## NONCE

El contador de la cuenta. Fija el orden y evita la repetición.

## MAXFEE · PRIORITY

Cuánto está dispuesto a pagar por unidad de gas, como máximo y como propina.

## GASLIMIT

El tope de trabajo. Si se agota, la ejecución se revierte.

## TO · VALUE · DATA

A quién, cuántos wei, y qué función con qué argumentos. Aquí data va vacío: es un pago simple.

## CÓMO SE DICE «LLAMA A ESTA FUNCIÓN»

Una llamada a contrato viaja en data: el selector de la función y después los argumentos, cada uno relleno a 32 bytes. Esta es una transacción real de Sepolia que envía 25 LINK de prueba:

[illegible]

## 01 SELECTOR · 4 BYTES

Los primeros 4 bytes de `keccak256("transfer(address,uint256)")`. Keccak de la Sesión 2, otra vez.

## 02 ARGUMENTO 1 · 32 BYTES

La dirección de destino, con 12 bytes de ceros a la izquierda para completar 32.

### 03 ARGUMENTO 2 · 32 BYTES

0x15af1d78b58c40000 =  $25 \times 10^{18}$ . LINK tiene 18 decimales: 25 tokens son 25 seguidos de 18 ceros.



# UN PROGRAMA QUE NO TERMINA NUNCA

Con un lenguaje Turing-completo, alguien puede desplegar un bucle infinito. Y no existe algoritmo que decida de antemano si un programa cualquiera terminará: es el problema de la parada, demostrado por Turing en 1936.

**Solución: no se pregunta si el programa termina. Se cobra cada paso por adelantado, y cuando se acaba lo pagado, se detiene.**

## QUÉ RESUELVE

- Nadie puede bloquear la red con un cómputo infinito.
- Quien consume recursos de miles de nodos los paga.
- El costo de un ataque de denegación de servicio se vuelve real.

## QUÉ NO RESUELVE

- No hace barato lo que de verdad es costoso.
- No evita que un contrato mal hecho consuma gas en vano.
- No protege al usuario que firma un límite demasiado alto sin mirar.

# EL LÍMITE DE GAS Y LO QUE PASA SI SE AGOTA

| CASO                        | QUÉ OCURRE CON EL ESTADO                                 | QUÉ OCURRE CON EL GAS                                   |
|-----------------------------|----------------------------------------------------------|---------------------------------------------------------|
| Termina bien                | Se aplican todos los cambios.                            | Se cobra solo lo usado. El resto del límite no se paga. |
| Se revierte (require falla) | No se aplica ningún cambio: como si no hubiera ocurrido. | Se cobra el gas consumido HASTA el punto del error.     |
| Se agota el límite          | No se aplica ningún cambio.                              | Se cobra el límite COMPLETO: el trabajo sí se hizo.     |

 VISTO EN SEPOLIA

Una llamada a transfer() firmada con límite de 500 000 falló: consumió 22 550 de gas y pagó 0,0000439 ETH por un resultado nulo. Revertir protege el estado, no el bolsillo.

Por eso las billeteras estiman el gas antes de enviar: ejecutan la transacción en simulación sobre el estado actual y proponen un límite con un margen.



# EIP-1559: TARIFA BASE MÁS PROPINA

Desde agosto de 2021 el precio no se subasta a ciegas. Tiene dos partes con destinos distintos:

## TARIFA BASE (BASEFEE)

La fija el protocolo en cada bloque, igual para todos. Si el bloque anterior usó más de la mitad de su capacidad, sube; si usó menos, baja. Como máximo un 12,5 % por bloque. Se QUEMA: no la recibe nadie.

## PROPINA (PRIORITYFEE)

La elige el usuario y la recibe el validador que incluye la transacción. Es el incentivo para ser incluido antes cuando hay congestión.

$\text{precio efectivo} = \min(\text{maxFeePerGas}, \text{tarifa\_base} + \text{maxPriorityFeePerGas})$        $\text{costo} = \text{gas\_usado} \times \text{precio efectivo}$

Lo que se firma es un máximo. Si la tarifa base baja entre la firma y la inclusión, la diferencia no se cobra.



# LA FÓRMULA, COMPROBADA EN SEPOLIA

$$\text{base\_siguiente} = \text{base} + \text{base} \times (\text{usado} - \text{objetivo}) \div \text{objetivo} \div 8$$

$$\text{objetivo} = \text{límite} \div 2 = 30\,000\,000$$

| BLOQUE     | TARIFA BASE      | GAS USADO DEL LÍMITE | FÓRMULA PREDICE  | BLOQUE SIGUIENTE, REAL | CAMBIO   |
|------------|------------------|----------------------|------------------|------------------------|----------|
| 11 727 239 | 1,029057347 gwei | 23,81 %              | 0,961679578 gwei | 0,961679578 gwei       | -6,55 %  |
| 11 727 240 | 0,961679578 gwei | 99,98 % (lleno)      | 1,081844398 gwei | 1,081844398 gwei       | +12,50 % |
| 11 727 241 | 1,081844398 gwei | 32,90 %              | 1,035597630 gwei | 1,035597630 gwei       | -4,27 %  |

Tres bloques seguidos, con límite de 60 millones de gas. Un bloque lleno sube la base casi exactamente el máximo (12,5 %); uno con poco uso la baja. La predicción coincide con el valor real hasta el último wei.

## LO QUE ESTO SIGNIFICA

La tarifa base no la decide nadie: se calcula. Por eso una billetera la puede predecir antes de firmar.



# CUÁNTO COSTÓ DE VERDAD UN PAGO EN SEPOLIA

| DATO                   | VALOR                                                                     | DE DÓNDE SALE                                        |
|------------------------|---------------------------------------------------------------------------|------------------------------------------------------|
| Gas usado              | 21 000                                                                    | Transferencia simple: solo la base (A.17).           |
| Tarifa base del bloque | 0,947200913 gwei                                                          | La fijó el protocolo (A.14).                         |
| Propina máxima firmada | 2 gwei                                                                    | La eligió la billetera.                              |
| Precio efectivo        | 2,947200913 gwei                                                          | Base + propina: no llegó al máximo de 4,05 gwei.     |
| Costo total            | $21\,000 \times 2,947200913 \text{ gwei} = 0,000061891219173 \text{ ETH}$ | Lo que salió de la cuenta, además del valor enviado. |
| Quemado                | $21\,000 \times 0,947200913 \text{ gwei} = 0,000019891219173 \text{ ETH}$ | Desaparece de la oferta total de ETH.                |
| Para el validador      | $21\,000 \times 2 \text{ gwei} = 0,000042 \text{ ETH}$                    | La propina.                                          |

Comprueben la suma: quemado + propina = costo total. Y el tope firmado era  $21\,000 \times 4,05 \text{ gwei} \approx 0,000085 \text{ ETH}$ : la diferencia no se cobró.

# LA EVM ES UNA MÁQUINA DE PILA

No tiene registros como un procesador: opera sobre una pila de palabras de 256 bits. Cada instrucción saca valores de arriba, opera y pone el resultado.

```
3 + 4, Y GUARDARLO  
  
PUSH1 0x03    // pila: [3]  
PUSH1 0x04    // pila: [4, 3]  
ADD          // pila: [7]  
PUSH1 0x00    // pila: [0, 7]  
SSTORE       // guarda 7 en la ranura 0 · pila: []
```

- Palabras de 256 bits: por eso uint256 es el tipo natural de Solidity.
- Profundidad máxima de 1 024 elementos.
- Unas 150 instrucciones, cada una con su costo fijo en gas.
- Solidity compila a esta secuencia de bytes: el bytecode que se despliega.

## NO HACE FALTA ESCRIBIR ESTO A MANO

Nunca van a programar en opcodes. Pero cuando en la Sesión 7 optimicen gas, o en la Sesión 9 lean un exploit, van a necesitar saber que cada línea de Solidity se paga en instrucciones como estas.



# LO QUE CUESTA CADA INSTRUCCIÓN

| INSTRUCCIÓN         | GAS                                   | QUÉ HACE                                                 |
|---------------------|---------------------------------------|----------------------------------------------------------|
| ADD · MUL           | 3 · 5                                 | Aritmética en la pila. Casi gratis.                      |
| KECCAK256           | 30 + 6 por palabra                    | Calcular un hash.                                        |
| SLOAD               | 2 100 primera vez · 100 después       | Leer una ranura del almacenamiento.                      |
| SSTORE 0 → x        | 20 000 (+ 2 100 primer acceso)        | Estrenar una ranura. La operación más cara de uso común. |
| SSTORE x → y        | 2 900 (+ 2 100 primer acceso)         | Cambiar un valor que ya existía.                         |
| SSTORE x → 0        | 2 900 (+ 2 100) y devuelve 4 800      | Liberar espacio tiene premio (hasta 1/5 del gas usado).  |
| LOG                 | 375 + 375 por tema + 8 por byte       | Emitir un evento.                                        |
| Transacción base    | 21 000                                | Solo por existir: firma, nonce, saldo.                   |
| Datos de la llamada | 4 por byte cero · 16 por byte no cero | Lo que viaja en data.                                    |

Vigentes desde Berlin (2021) y London (reembolsos). La actualización Glamsterdam, prevista para Sepolia en octubre de 2026, cambia varios de estos precios.



# STORAGE, MEMORY Y CALldata

| LUGAR    | DURA                               | QUIÉN LO MODIFICA          | COSTO                                                 |
|----------|------------------------------------|----------------------------|-------------------------------------------------------|
| storage  | Para siempre, entre transacciones. | Solo el propio contrato.   | Muy caro: es el estado que replican todos los nodos.  |
| memory   | Solo durante la llamada.           | La función que se ejecuta. | Barato y crece con el tamaño usado.                   |
| calldata | Solo durante la llamada.           | Nadie: es de solo lectura. | El más barato para parámetros: no se copia.           |
| stack    | Solo durante la instrucción.       | La propia EVM.             | Casi gratis, pero limitado a 16 variables accesibles. |

```
function sumar(uint256[] calldata datos) // no se copia: 39 360 de gas (50 números)
function sumar(uint256[] memory datos)   // se copia antes: 42 058 de gas
```

Error típico: creer que memory «guarda» algo. Al terminar la llamada desaparece; si el dato debe sobrevivir, tiene que ir a storage, y eso se paga.



# DE DÓNDE SALE CADA UNIDAD DE GAS

| OPERACIÓN MEDIDA                        | BASE   | DATOS | INSTRUCCIÓN CARA     | RESTO | TOTAL  |
|-----------------------------------------|--------|-------|----------------------|-------|--------|
| <code>escribirNueva(42) · 0 → 42</code> | 21 000 | 204   | SSTORE nueva 22 100  | 298   | 43 602 |
| <code>sobrescribir(43) · 42 → 43</code> | 21 000 | 204   | SSTORE 5 000         | 275   | 26 479 |
| <code>leerEnTransaccion()</code>        | 21 000 | 64    | SLOAD frío 2 100     | 200   | 23 364 |
| <code>borrar() · 43 → 0</code>          | 21 000 | 64    | SSTORE 5 000 – 4 800 | 181   | 21 445 |
| <code>registrarConEvento(42)</code>     | 21 000 | 204   | LOG2 1 381           | 277   | 22 862 |

Datos:  $204 = 31 \text{ bytes en cero} \times 4 + 5 \text{ bytes distintos de cero} \times 16$ . El «resto» es el trabajo de Solidity: comparar el selector, decodificar el argumento, saltar a la función.



## PREGUNTA PARA EL LABORATORIO

¿Por qué borrar cuesta casi lo mismo que una transferencia simple?

# DE LA FIRMA A LA FINALIDAD

- |           |                     |                                                                                                                        |
|-----------|---------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>01</b> | <b>FIRMA</b>        | La billetera arma la transacción, estima el gas y la firma con la clave privada. Nada ha salido del computador.        |
| <b>02</b> | <b>MEMPOOL</b>      | Se envía a un nodo, que la valida y la propaga por rumor (Sesión 3). Queda pendiente, a la vista de todos.             |
| <b>03</b> | <b>INCLUSIÓN</b>    | Un constructor de bloques la elige, normalmente por propina. El validador de turno propone el bloque cada 12 segundos. |
| <b>04</b> | <b>CONFIRMACIÓN</b> | El bloque se propaga y otros validadores lo atestiguan. Aparece en el explorador con estado y recibo.                  |
| <b>05</b> | <b>FINALIDAD</b>    | Unas dos épocas después, unos 13 minutos, queda finalizada (Sesión 4).                                                 |

Error típico: «si ya salió en el explorador, es definitiva». Confirmada no es lo mismo que finalizada: lo vimos con las reorganizaciones de la Sesión 4.

# LEER UNA TRANSACCIÓN EN EL EXPLORADOR

| CAMPO DEL EXPLORADOR | QUÉ SIGNIFICA                                               | ERROR TÍPICO AL LEERLO                                                    |
|----------------------|-------------------------------------------------------------|---------------------------------------------------------------------------|
| Status               | Success o Fail.                                             | Creer que Fail no costó nada.                                             |
| From · To            | Quién firmó y a quién iba.                                  | En un token, To es el CONTRATO del token, no quien recibe.                |
| Value                | ETH enviado con la llamada.                                 | Pensar que 0 ETH significa que no se movió valor: los tokens van en data. |
| Transaction Fee      | Lo que pagó el remitente.                                   | Confundirlo con el valor transferido.                                     |
| Gas Limit & Usage    | El tope firmado y lo consumido.                             | Asumir que se cobra el límite cuando la transacción terminó bien.         |
| Burnt Fees           | La parte quemada de la tarifa base.                         | Creer que la recibió el validador.                                        |
| Input Data           | El campo data, decodificado si el contrato está verificado. | No ver nada útil porque el contrato no está verificado.                   |
| Logs                 | Los eventos emitidos.                                       | Ignorarlos: son la forma más confiable de saber qué pasó.                 |



# SEIS IDEAS PARA LA UNIDAD II

01

Ethereum es una máquina de estados: mismo estado más misma transacción da el mismo resultado en todo el mundo.

---

02

Solo una cuenta externa inicia algo. Un contrato nunca actúa por su cuenta.

---

03

Una llamada a función es un selector de 4 bytes seguido de argumentos de 32 bytes.

---

04

El gas existe porque no se puede saber si un programa termina: se cobra cada paso.

---

05

Costo = gas usado × (tarifa base quemada + propina). Lo firmado es un máximo.

---

06

Escribir en storage es, con diferencia, lo más caro. Diseñar contratos es diseñar alrededor de eso.

---



# PREGUNTA DE VERIFICACIÓN

**Una transferencia de token muestra en el explorador «Value: 0 ETH» y «Status: Fail». ¿Qué afirmación es correcta?**

- A** No se movió nada y no costó nada: el valor es cero y además falló.
- B** No se aplicó ningún cambio de estado, pero el remitente pagó el gas consumido hasta el error.
- C** Se transfirieron los tokens, pero no el ETH, porque el Value es cero.

Un minuto en parejas y respuesta a mano alzada. Después, a abrir la billetera.

# B

Q APROXIMADAMENTE 60 MINUTOS · EN PAREJAS

## LABORATORIO 05

Primero se lee la cadena como un forense. Después se predice y se mide el costo de cada instrucción.



# PUNTO DE CONTROL DE BILLETERAS

## 01 ABRIR

Abrir la billetera creada en la Sesión 2 y desbloquearla.

## 02 RED

Cambiar a la red de prueba Sepolia. Confirmar que la red NO es la principal.

## 03 SALDO

Verificar que hay ETH de prueba. Si no, avisar: se reparte desde la billetera institucional.

## 04 FRASE

Confirmar que la frase de recuperación está guardada en papel, fuera del computador.



### QUIEN PERDIÓ LA BILLETERA LA REHACE HOY

No en la Sesión 6, que es cuando se despliega el primer contrato. Nunca se usa esta billetera con dinero real, y la frase no se comparte con nadie, tampoco con el docente.

# ANÁLISIS FORENSE DE TRES TRANSACCIONES

En el explorador de Sepolia, cada pareja encuentra tres transacciones reales de tipos distintos y llena una hoja de trabajo por cada una.

| TIPO                              | CÓMO ENCONTRARLA                                                       | QUÉ TIENE QUE IDENTIFICAR                                                               |
|-----------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 1 · <b>Transferencia de ETH</b>   | La dirección de su propia billetera, si recibió fondos de prueba.      | Gas usado exactamente 21 000. Valor, tarifa base, propina, costo total.                 |
| 2 · <b>Transferencia de token</b> | El contrato de LINK o USDC de prueba → pestaña Token Transfers.        | Selector a9059cbb, destinatario real en los argumentos, el evento Transfer en los logs. |
| 3 · <b>Llamada que falló</b>      | Un contrato con mucho uso: buscar una fila con el ícono rojo de error. | Por qué falló, cuánto gas consumió y cuánto costó igualmente.                           |

 LA PREGUNTA QUE TIENE QUE RESPONDER EL INFORME

Para cada una: ¿cuánto se quemó, cuánto recibió el validador, y por qué el gas usado fue el que fue?

# QUÉ SE ABRE Y DÓNDE SE MIRA

- |           |                     |                                                                                                                                |
|-----------|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>01</b> | <b>BUSCAR</b>       | sepolia.etherscan.io → pegar el hash, la dirección o el contrato en la barra de búsqueda. Comprobar que la URL dice «sepolia». |
| <b>02</b> | <b>OVERVIEW</b>     | Status, Block, From, To, Value, Transaction Fee, Gas Price. Anotarlos tal cual, con todas sus cifras.                          |
| <b>03</b> | <b>MÁS DETALLES</b> | «Click to show more»: Gas Limit & Usage, Gas Fees (Base, Max, Max Priority), Burnt Fees, Input Data.                           |
| <b>04</b> | <b>INPUT DATA</b>   | «Decode Input Data»: función y argumentos. Si no decodifica, el contrato no está verificado.                                   |
| <b>05</b> | <b>LOGS</b>         | Pestaña Logs: cada evento con su nombre, sus temas (topics) y sus datos.                                                       |

Salida esperada: tres hojas de trabajo llenas, con los hashes pegados como enlaces que abran.



# UNA HOJA DE TRABAJO BIEN LLENA

| CAMPO                   | TRANSFERENCIA DE USDC · BLOQUE 11 727 240                                    |
|-------------------------|------------------------------------------------------------------------------|
| Hash                    | 0xe51f75bad82aceb9e03cbf9af64d080df91203c43f801b99f7d71556654fbf0a           |
| To · Value              | El contrato de USDC (no la persona) · 0 ETH                                  |
| Gas Limit · Used        | 69 301 · 62 171 (89,7 % del límite)                                          |
| Base · efectivo         | 0,961679578 gwei · 0,962779578 gwei (propina 0,0011 gwei)                    |
| Fee = quemado + propina | 0,000059856969143838 = 0,000059788581043838 + 0,0000000683881 ETH            |
| Input Data              | transfer · selector a9059cbb · destinatario 0x9d7f...25db · monto 18 100 000 |
| Logs                    | 1 evento Transfer: 18,1 USDC (el token tiene 6 decimales)                    |

Fíjense: la propina es casi nula y el bloque estaba lleno. Por eso el bloque siguiente subió su tarifa base un 12,5 % (A.14).



# PREDECIR PRIMERO, MEDIR DESPUÉS

- |           |                          |                                                                                                         |
|-----------|--------------------------|---------------------------------------------------------------------------------------------------------|
| <b>01</b> | <b>PREDECIR</b>          | Con A.17 y A.19, escribir cuánto gas cuesta cada operación del contrato Operaciones. Sin correr nada.   |
| <b>02</b> | <b>PROBAR EL ENTORNO</b> | Correr las pruebas de la sesión: 7 en verde significan que Hardhat funciona.                            |
| <b>03</b> | <b>MEDIR</b>             | Correr el script: imprime una tabla de 12 filas, en la red local, sin gastar ETH ni necesitar internet. |
| <b>04</b> | <b>COMPARAR</b>          | Anotar la diferencia entre predicción y medición, y explicar las dos mayores.                           |

EN LA CARPETA LABORATORIOS-EVM

```
npx hardhat test test/s05/Operaciones.test.js    # → 7 passing
npx hardhat run scripts/s05/medir-gas.js          # → la tabla
  Escribir en una ranura vacía (0 → 42)           43.602
  Sobrescribir una ranura ocupada (42 → 43)       26.479
```



# LO QUE SUELE SALIR MAL

| SÍNTOMA                                               | CAUSA                                                                                  |
|-------------------------------------------------------|----------------------------------------------------------------------------------------|
| El explorador no encuentra mi dirección               | Están en el explorador de la red principal y no en el de Sepolia.                      |
| «Value: 0» en una transferencia de token              | Correcto: el token no es ETH. El monto está en Input Data y en los logs.               |
| El Input Data es ilegible                             | El contrato no está verificado. Busquen otro token que sí lo esté.                     |
| El monto del token parece enorme                      | No dividieron por los decimales: USDC usa 6, LINK usa 18.                              |
| npm install falla en la sala                          | Restricción de red. Plan B: el docente comparte la carpeta node_modules por red local. |
| El script da cifras algo distintas a las de la lámina | Otra versión del compilador u optimizador. Las proporciones deben mantenerse.          |

Si algo no se resuelve en 5 minutos, se anota en la hoja y se sigue con la siguiente parte. No se pierde el laboratorio por un problema de red.



# PREGUNTA DE VERIFICACIÓN

**Antes de cerrar el computador: su hoja dice «Fee = 0,00012 ETH» y «Burnt + propina = 0,00009 ETH». ¿Qué hacen?**

- A** Nada: son aproximaciones y el explorador redondea.
- B** Revisar: la suma tiene que coincidir exactamente; algo se copió mal o se usó el máximo firmado en vez del precio efectivo.
- C** Cambiar el Fee para que cuadre con la suma.

Terminado = tres hashes que abren + sumas que cuadran + tabla de gas con las dos diferencias explicadas.



# EVIDENCIA DEL LABORATORIO 05

| PARTE       | CONTENIDO                                                                                            | PESO |
|-------------|------------------------------------------------------------------------------------------------------|------|
| A · forense | Tres hojas de trabajo con hashes que abren, las sumas comprobadas y las cuatro preguntas de la guía. | 50 % |
| B · gas     | La tabla con predicciones, mediciones y la explicación de las dos mayores diferencias.               | 35 % |
| Reflexión   | Un párrafo: ¿qué operación evitarían en un contrato real después de hoy, y por qué?                  | 15 % |

Máximo dos páginas, con la plantilla de informe de la guía. Un informe por pareja. Se sube a la plataforma antes de la Sesión 6.



## POLÍTICA DE LABORATORIOS

Sin hashes de transacción verificables, la parte A no se califica. Y los hashes tienen que abrir en el explorador: una transacción inventada por un asistente de IA no existe en la cadena, y eso se comprueba en diez segundos.



# LA PREGUNTA DE LA SEMANA

**Si guardar un dato en la cadena cuesta decenas de miles de gas, ¿qué NO guardarían nunca en un contrato, y dónde lo guardarían en su lugar?**

## PARA LA PRÓXIMA SESIÓN

- Entregar el informe del laboratorio 05 antes de la Sesión 6.
- Leer la introducción de la documentación oficial de Solidity: «Introduction to Smart Contracts».
- Terminar el anteproyecto: se entrega en la Sesión 6. Máximo tres páginas, con la plantilla publicada.
- Opcional: jugar las dos primeras lecciones de CryptoZombies.



LO QUE SE LLEVAN DE HOY

# Cada instrucción tiene precio. Programar bien es saber cuánto cuesta lo que se escribe.

Hoy leyeron la máquina desde fuera. La próxima sesión escriben su primer programa para ella y lo despliegan en una red pública.

SESIÓN 6 · SOLIDITY I · PRIMER CONTRATO EN SEPOLIA · ENTREGA DEL ANTEPROYECTO